

Contagem paralela de objetos em uma matriz binária

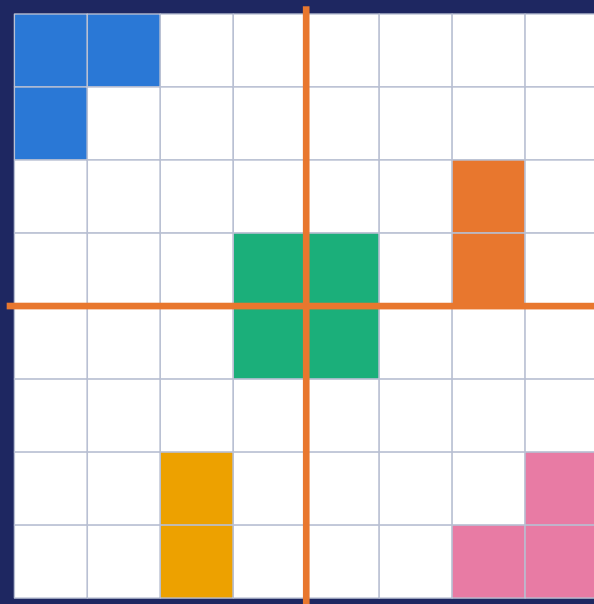
Versão sequencial e versão paralela com POSIX Threads

Lucas Gaelzer Machado

23200069

Stefan Chagas

23200106



Exemplo 3 do enunciado: 5 objetos, grade 2×2

O problema: contar objetos com conectividade 8

0 = fundo, 1 = primeiro plano.

Um objeto é um grupo de células 1 ligadas na horizontal, na vertical ou na diagonal.

Cada célula tem 8 vizinhos.

Sem as diagonais (conectividade 4), um tabuleiro de xadrez 9×9 teria 41 objetos; com conectividade 8, é um só.

Desafio da versão paralela

Um objeto pode atravessar as divisões da matriz.
Somar as contagens de cada região conta o mesmo objeto mais de uma vez.

1	1	0	0	0
1	1	0	0	0
0	0	0	1	0
0	0	0	1	0
1	0	0	0	0

Exemplo 1 (5×5): 3 objetos

Estratégia: POSIX Threads

Processos (fork)

- Memória isolada: rótulos e equivalências precisariam de mmap ou pipes
- Criação mais cara
- Mais recursos para liberar

Pthreads ✓ escolhida

- Matriz e rótulos já compartilhados
- Criação barata
- Risco: condições de corrida → mutex e particionamento dos dados

Ideia central: dividir a matriz em blocos, rotular cada bloco em paralelo e depois unir os pedaços de objetos que se tocam nas fronteiras.

Versão sequencial: flood fill iterativo

- 1 Percorre a matriz linha a linha**
Vetor de rótulos zerado marca o que já foi visitado.
- 2 Célula 1 sem rótulo → novo objeto**
Incrementa o contador e empilha a célula.
- 3 Desempilha e testa os 8 vizinhos**
Vizinho 1 e não visitado recebe o rótulo ao ser empilhado (nunca entra duas vezes).
- 4 Pilha explícita no heap**
Sem recursão: objetos com milhões de células não estouram a pilha de chamadas.

-1,-1	-1,+0	-1,+1
+0,-1	(r,c)	+0,+1
+1,-1	+1,+0	+1,+1

■ horizontal/vertical ■ diagonal

Tempo $O(L \cdot C)$

cada célula é empilhada no máximo uma vez

Decomposição: grade de blocos e fila dinâmica

B0 T0	B1 T1	B2 T2	B3 T3
B4 T1	B5 T0	B6 T3	B7 T2
B8 T2	B9 T3	B10 T0	B11 T1
B12 T3	B13 T2	B14 T1	B15 T0

Ex.: -t 2 → grade 4×4 (padrão 2T x 2T). Cores = thread que retirou o bloco

Blocos retangulares

-b LxC configura a grade; padrão 2T x 2T. Faixa i começa em $[i \cdot n/B]$, então as sobras se espalham.

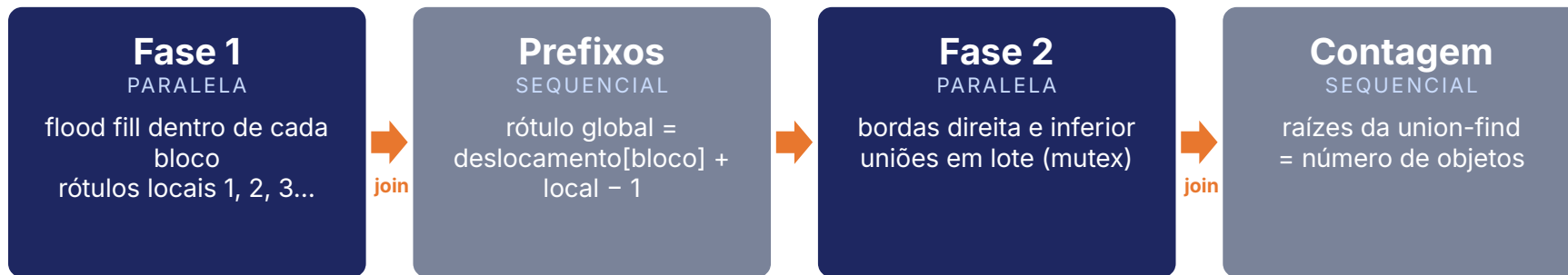
Fila dinâmica

Um contador proximo_bloco protegido por mutex_fila. Cada thread pega o próximo bloco livre até esvaziar.

Por quê?

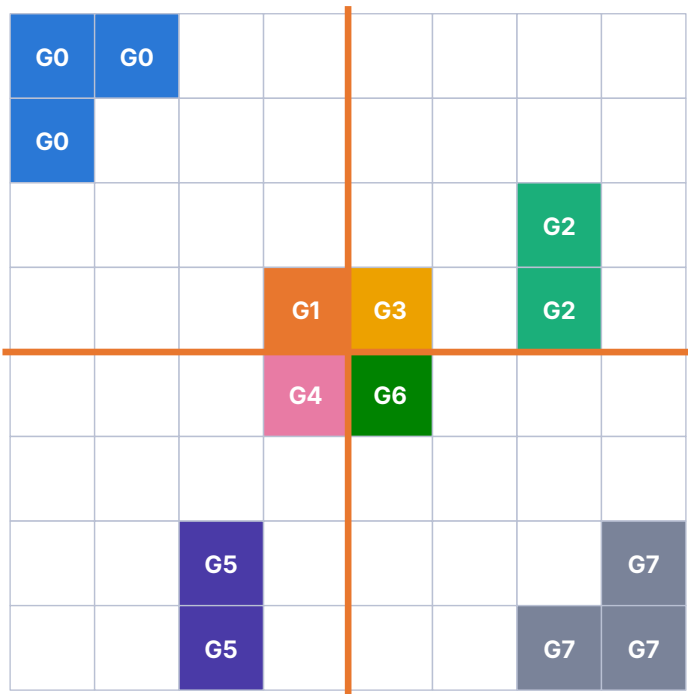
Mais blocos que threads equilibra regiões densas e esparsas; blocos têm menos borda que faixas.

Fases e sincronização



Matriz	somente leitura
Rótulos	cada bloco é escrito por uma única thread; a fase 2 só lê
Fila de blocos	mutex_fila em volta do contador
Union-find	mutex_uniao, uma vez por lote de até 4096 pares
Deadlock	impossível: nenhuma thread segura dois mutexes

Consolidação: por que somar não basta



Exemplo 3, grade 2×2, antes da consolidação

8

soma simples dos componentes
locais

5

objetos de verdade

O objeto central de 2×2 cai nos 4 blocos e vira G1, G3, G4 e G6.

Rótulos globais únicos sem coordenação: depois da fase 1, uma soma de prefixos sobre os componentes de cada bloco dá a cada bloco uma faixa própria de rótulos.

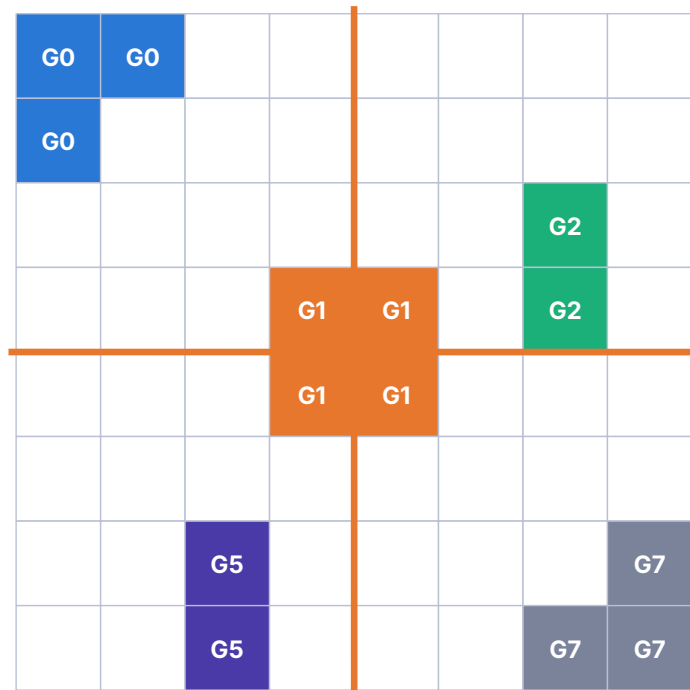
Fronteiras, diagonais e o encontro de 4 blocos

Cada bloco examina só a borda direita e a inferior, então cada fronteira é vista uma vez.

Cada célula 1 da borda testa 3 vizinhos do outro lado: em frente e as duas diagonais. Isso cobre também a quina onde 4 blocos se encontram.

```
G1 ~ G3  horizontal  (3,3)-(3,4)
G1 ~ G4  vertical    (3,3)-(4,3)
G1 ~ G6  diagonal    (3,3)-(4,4)
G3 ~ G4  antidiagonal (3,4)-(4,3)
... 8 pares → 3 uniões efetivas
```

Union-find: a menor raiz vira o representante, então a ordem das uniões não muda o resultado.



Depois da união: G0, G1, G2, G5, G7 → 5 objetos

Demonstração

```
$ make
$ ./conta-objetos-sequencial tests/obrigatorios/exemplo3.txt
Objetos: 5
$ ./conta-objetos-paralelo -t 2 -b 2x2 -v tests/obrigatorios/exemplo3.txt
Componentes locais (soma simples): 8
[thread 0] equivalencia G1 ~ G3 ...
Objetos: 5
$ make test
== Resultado: 743 verificacoes, 0 falha(s)
$ ./conta-objetos-paralelo -t 4 -v -g 8000 8000 0.30 2026
Tempo por etapa (ms): ... fase 1 (paralela) 154.356 | prefixos e
union-find 2.045 | fase 2 (paralela) 0.649 | contagem 0.856
```

Testes de correção

743

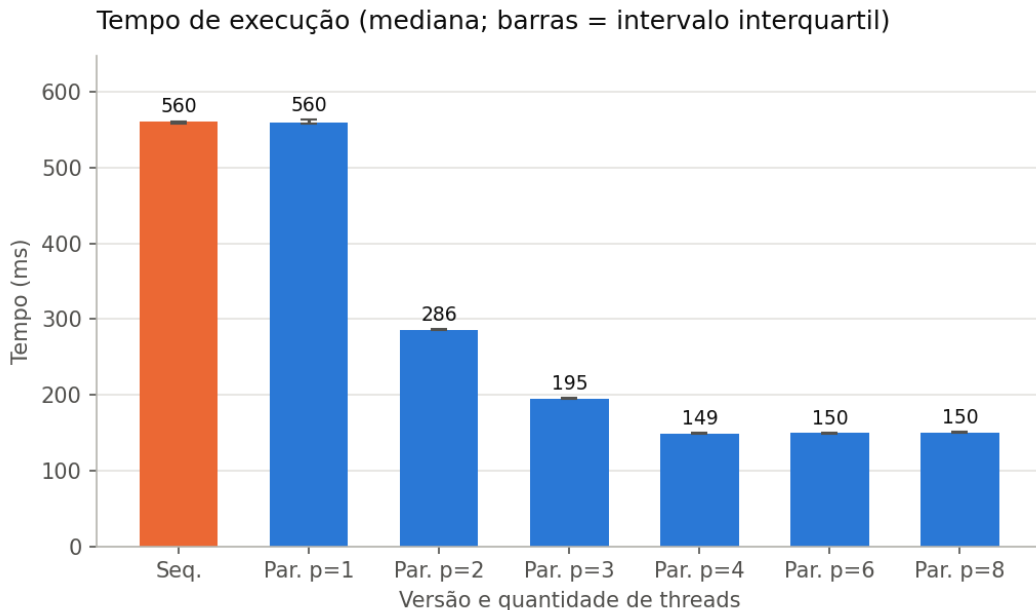
verificações, 0 falhas

- 5 obrigatórias + 10 adicionais $\times$ 1, 2, 3, 4, 8 threads $\times$ 7 grades
- 50 repetições: sempre o mesmo resultado
- 200 matrizes aleatórias: sequencial = paralela
- Oráculo independente em Python
- ThreadSanitizer, Helgrind, Valgrind: 0 erros

Ex.	Dimensões	Esperado	Sequencial	Paralelo
1	5×5	3	3	3
2	6×8	4	4	4
3	8×8	5	5	5
4	9×12	6	6	6
5	12×12	7	7	7

Casos adicionais: só zeros, serpentina por todos os blocos, tabuleiro de xadrez (só diagonais), X cruzando a quina, 1×20 , 14×1 , 1×1 ...

Desempenho: matriz 8000 × 8000



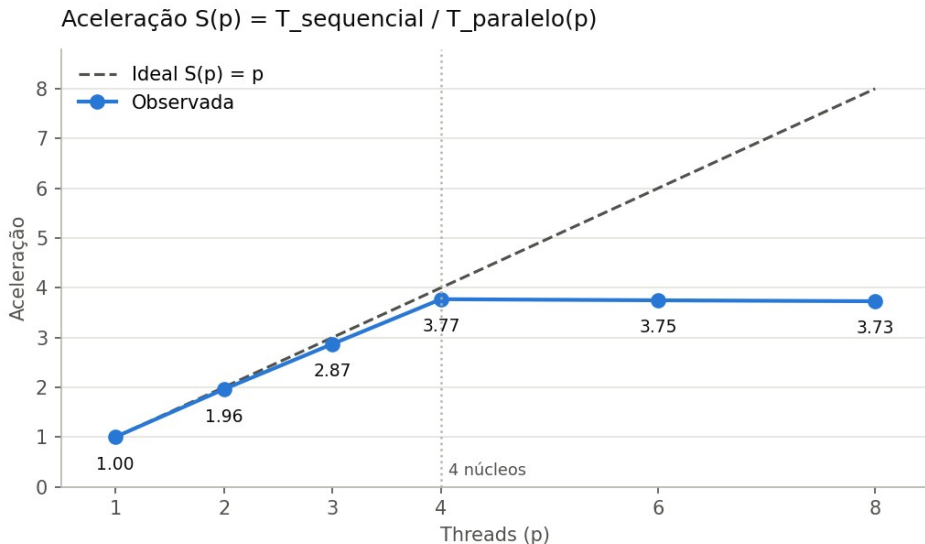
Mediana de 10 repetições intercaladas; 4 núcleos; 3.023.082 objetos

3,77×
aceleração com 4 threads

0,94
eficiência com 4 threads

560 → 149 ms
sequencial → paralela

Escalabilidade e quando o paralelo perde



Até 4 threads: quase linear. Fase 1 = 98% do tempo; consolidação < 1%.

6 e 8 threads: patamar de cerca de 3,75. Só há 4 núcleos; as threads extras dividem os mesmos núcleos.

Amdahl: cerca de 0,55% sequencial → máximo de 3,94 com 4 núcleos.

Exemplo 5 (12 × 12): sequencial 0,003 ms; paralela 0,073 ms (1 thread) a 0,224 ms (8 threads). Até 75× mais lenta: criar e esperar threads custa mais que contar 144 células.

Conclusões

1

Correto

Sequencial e paralela idênticas em 743 verificações, com conectividade 8 inclusive nas quinas.

2

Rápido onde importa

Aceleração de 3,77 com 4 threads em 8000×8000 ; nas matrizes minúsculas a sobrecarga domina.

3

O difícil é juntar

Dividir é simples. Consolidar exige rótulos únicos, diagonais e uma union-find protegida.

4

Próximo passo

União sem bloqueio com atômicos (C11) e paralelizar a soma de prefixos.